

Ottodot Full-Stack Engineer Take-Home

Trial Booking Reliability | AI tools welcome | Timebox: 3-4 hours

Please cap your time at 4 hours. If you run out of time, leave notes on what you would do next.

We encourage you to use AI tools. We are interested in how you steer it, question it, and ship something great.

Context

Ottodot runs live online science and math classes for kids. Parents need to book and pay a trial class for their child, and our team needs an accurate roster before class starts.

Trial classes are capped at 4 students per class.

Your task is to build the smallest working slice of a trial booking system that behaves correctly under important edge cases.

What To Build

Implement trial booking only. Do not implement regular enrollment.

Your solution should allow:

1. A parent to choose a child and pick an available trial class.
2. A parent to submit a trial booking.
3. A mock payment step or payment result to be recorded.
4. The booking status to be shown after submission.
5. An admin or teacher to see the trial class roster or a simple roster API/output.

Your solution must prevent or handle:

- duplicate confirmed bookings for the same child and class
- overbooking beyond 4 confirmed students
- payment failure without incorrectly adding the child to the confirmed roster
- the last-seat race condition described below

A polished frontend is not required. A simple UI is useful, but we care more about the data model, backend logic, invariants, tests, and your explanation. A CLI, script, API endpoint, server action, or minimal app is fine if it shows your thinking clearly.

Required Technical Scenario: Last-Seat Race

Please explicitly handle or explain this scenario:

1. User A selects the last available slot and moves to payment.
2. User B selects the same slot.
3. User B completes payment first and confirms the booking.
4. User A then tries to complete payment.

Your implementation must ensure that at most one user can end up with a confirmed booking for the last available seat.

In your README, explain:

- the approach you chose
- why you chose it
- what tradeoffs you accepted

Backend Design Requirements

In your README, include a short backend/design section covering:

- your data model or schema
- the key API endpoints, server actions, or backend functions
- booking statuses used, such as `pending_payment`, `confirmed`, `payment_failed`, `cancelled`, or similar
- how you prevent duplicate bookings
- how you handle payment failure
- how you handle two users competing for the last seat
- which checks belong in the UI, backend, database, or background job

Suggested Model

Use a small synthetic dataset. You can create it in any format you prefer: seed file, JSON, CSV, SQLite, Postgres, Supabase, or in-memory data.

Useful concepts may include:

- `parents`
- `students`
- `trial_classes`
- `bookings`
- `payment_attempts`

You may add fields if useful, but keep the model small.

Seed Data And Edge Cases

Please include seed data or setup steps so we can run the demo quickly.

Include enough cases to show:

- a class with available seats
- a class with exactly 3 confirmed students
- a duplicate booking attempt for the same child and class
- a payment failure case

What To Submit

1. Please submit a link to a public GitHub repo containing your implementation containing:
 - README.md
 - your implementation
 - synthetic data or setup instructions
 - tests or clear verification steps
 - AI_USAGE.md
 - *Note: Zip files are not accepted.*
2. A link to a short video walkthrough (5–8 minutes) — screen recording of you running the solution and briefly explaining your approach, the last-seat race handling, and any tradeoffs. Loom, YouTube (unlisted), or similar is fine.

README

Your README should explain:

- how to run your solution
- what you built
- time spent
- assumptions you made
- key architecture and backend decisions
- what you deliberately cut
- what you would monitor after release
- what you would do next with more time

AI_USAGE.md

Your AI_USAGE.md should explain:

- which AI tools you used
- what you used AI for
- one place where AI helped you move faster
- one place where you disagreed with, corrected, or rejected AI output
- what you would change about your AI workflow if you had to do this again
- how you verified the final implementation

Evaluation

We will look for:

- backend and data-model judgment
- correctness under payment and double-booking edge cases
- a working full-stack or backend-led flow
- sensible tests or verification
- clear scope control
- clear communication

Please prioritize correct backend behavior, clear edge-case handling, and verification over frontend polish or feature breadth.